

- [EasySplit](#)
 - [Sobre o projeto](#)
 - [Funcionalidades](#)
 - [Stack](#)
 - [Estrutura do projeto](#)
 - [Como rodar](#)
 - [1. Pré-requisitos](#)
 - [2. Configuração do ambiente](#)
 - [3. Variáveis de ambiente](#)
 - [4. Inicializar o banco de dados](#)
 - [5. Rodar o servidor de desenvolvimento](#)
 - [Testes](#)
 - [Comandos úteis](#)
 - [Arquitetura](#)
 - [Filtros Jinja2 customizados](#)

EasySplit

Sistema web para divisão de despesas compartilhadas entre grupos de amigos, repúblicas, viagens e eventos.

Sobre o projeto

O EasySplit permite que usuários criem grupos, cadastrem participantes, registrem despesas e visualizem automaticamente quanto cada pessoa deve pagar ou receber. Os saldos são calculados de forma automática e o sistema sugere acertos simples entre os participantes.

Funcionalidades

- Cadastro, login e logout de usuários
- Criação e listagem de grupos
- Gerenciamento de participantes por grupo
- Registro de despesas com pagador e divisão igualitária
- Cálculo automático de saldos individuais

- Sugestão de acertos entre participantes
- Histórico de despesas por grupo
- Dashboard com resumo financeiro do grupo

Stack

Camada	Tecnologia
Backend	Python 3 + Flask 3
ORM	SQLAlchemy + Flask-SQLAlchemy
Formulários	Flask-WTF + WTForms
Auth	Flask-Login + Werkzeug
Frontend	Jinja2 + Bootstrap + JS pontual
Banco	SQLite (easysplit.db)
Testes	pytest + pytest-flask

Estrutura do projeto

```
easysplit/  
├─ app.py                # Application factory, filtros Jinja2, CLI init-db  
├─ extensions.py         # Instâncias de db e login_manager  
├─ requirements.txt  
├─ models/               # Models SQLAlchemy (User, Group, Participant,  
Expense...)  
├─ routes/               # Blueprints: auth, groups, notifications, oauth  
├─ services/             # Regras de negócio: cálculo de saldos e divisão  
├─ forms/                # Formulários WTForms  
├─ templates/            # Templates Jinja2 + Bootstrap  
├─ static/               # CSS customizado  
└─ tests/                # Testes automatizados com pytest
```

Como rodar

1. Pré-requisitos

- Python 3.10 ou superior
- pip

2. Configuração do ambiente

```
# Criar e ativar ambiente virtual
python -m venv venv

# Windows
venv\Scripts\activate

# Linux/macOS
source venv/bin/activate

# Instalar dependências
pip install -r requirements.txt
```

3. Variáveis de ambiente

Crie um arquivo `.env` na raiz do projeto:

```
SECRET_KEY=sua-chave-secreta-aqui
FLASK_ENV=development
```

Sem o arquivo `.env`, a aplicação usa `SECRET_KEY="dev"` por padrão — **não use em produção**.

4. Inicializar o banco de dados

```
flask --app app init-db
```

Isso cria o arquivo `easysplit.db` na raiz do projeto.

5. Rodar o servidor de desenvolvimento

```
flask --app app run --debug
```

Acesse em: <http://localhost:5000>

Testes

```
# Rodar todos os testes
pytest

# Com saída detalhada
pytest -v

# Arquivo específico
pytest tests/test_auth.py
```

Comandos úteis

```
# Verificar rotas registradas
flask --app app routes

# Reinicializar banco (apaga e recria)
flask --app app init-db
```

Arquitetura

A aplicação segue o padrão **Application Factory** com **Blueprints**:

- `create_app()` em `app.py` inicializa extensões, registra blueprints e filtros Jinja2
- Extensões (`db`, `login_manager`) centralizadas em `extensions.py`
- Regras de negócio isoladas em `services/`:
 - `settlement_service.py` — cálculo de saldos e sugestões de acerto
 - `expense_split_service.py` — divisão igualitária com centavos corretos
- Validação dupla: formulário WTForms + backend

Filtros Jinja2 customizados

Filtro	Exemplo de uso	Saída
brl	{{ 42.5 brl }}	R\$ 42,50
date_br	{{ data date_br }}	29/05/2026